

# Mean, covariance, eigenvectors: the mathematics of PCA

The statistics and linear algebra behind the PCA lecture and Assignments 1–2, with the numbers written out

Fall 2026

---

## Who this is for

The PCA lecture paired every formula with a picture and left the algebra to one slide. This handout does the algebra, one object at a time, on a data set of four points in two dimensions so that every number can be checked by hand. It goes a little beyond the lecture in three places: it derives why the top eigenvector is the direction of greatest variance, it explains where the singular value decomposition comes in, and it works out what standardization and whitening do to the same four points. The NumPy line after each definition is the one that Assignment 2's `fit_pca` helper uses. Cover each calculation and try it before reading on; the answers to “Try it” are at the end.

**PCA in one sentence.** Centre the data, form the covariance matrix, take its eigenvectors, sort them by eigenvalue. The eigenvectors are the directions along which the cloud is spread; the eigenvalues are the variances along those directions.

**The running example.** Four responses to four stimuli, each a vector of two entries (two neurons, or two pixels):

$$\mathbf{x}^{(1)} = (5, 3), \quad \mathbf{x}^{(2)} = (1, -1), \quad \mathbf{x}^{(3)} = (4, 0), \quad \mathbf{x}^{(4)} = (2, 2).$$

Stacked as rows they form the  $4 \times 2$  response matrix  $\mathbf{X}$ , stimuli down, features across, the convention used throughout the course.

## 1 The statistics of one variable

### 1.1 Mean and deviations

The *mean* of  $n$  numbers is their sum divided by  $n$ . The first feature (the first column of  $\mathbf{X}$ ) has values 5, 1, 4, 2:

$$\bar{x}_1 = \frac{1}{4}(5 + 1 + 4 + 2) = 3, \quad \bar{x}_2 = \frac{1}{4}(3 - 1 + 0 + 2) = 1.$$

The *mean vector*  $\boldsymbol{\mu} = (\bar{x}_1, \bar{x}_2) = (3, 1)$  is the average response, one point standing in for all four. Subtracting it from every row *centres* the data:

$$\tilde{\mathbf{x}}^{(1)} = (2, 2), \quad \tilde{\mathbf{x}}^{(2)} = (-2, -2), \quad \tilde{\mathbf{x}}^{(3)} = (1, -1), \quad \tilde{\mathbf{x}}^{(4)} = (-1, 1).$$

The centred rows sum to zero in each column, always: that is what subtracting the mean does. Everything that follows is computed on  $\tilde{\mathbf{X}}$ , and the mean is kept aside so that it can be added back at the end.

In NumPy: `mu = X.mean(axis=0); Xc = X - mu`

## 1.2 Variance and standard deviation

The *variance* of a variable is the mean of its squared deviations from its mean:

$$\text{var}(x_1) = \frac{1}{n} \sum_{i=1}^n (x_1^{(i)} - \bar{x}_1)^2 = \frac{1}{4}(4 + 4 + 1 + 1) = 2.5, \quad \text{var}(x_2) = \frac{1}{4}(4 + 4 + 1 + 1) = 2.5.$$

The *standard deviation*  $\sigma$  is its square root,  $\sigma_1 = \sigma_2 = \sqrt{2.5} \approx 1.58$ , and has the units of the data; the variance has those units squared.

$n$  or  $n - 1$ . Software usually divides by  $n - 1$  rather than  $n$  (NumPy's `np.var` divides by  $n$  unless told `ddof=1`; `np.cov` and Assignment 2's `fit_pca` divide by  $n - 1$ ). The  $n - 1$  version is the unbiased estimate of the variance of the population the sample came from; the  $n$  version is the variance of the sample itself. With  $n - 1$  every variance and covariance in this handout is multiplied by  $4/3$ , and *nothing else changes*: the eigenvectors are identical and the eigenvalues all scale by the same factor. The lecture slides use  $1/n$  because the formulas read more simply.

## 1.3 z-scores

A z-score rescales a variable so that its mean is 0 and its standard deviation is 1:  $z^{(i)} = (x^{(i)} - \bar{x})/\sigma$ . For the first feature,  $z_1 = (2, -2, 1, -1)/1.58 = (1.26, -1.26, 0.63, -0.63)$ . A z-score of 1.26 means “1.26 standard deviations above the mean”. Section 6 shows what happens to PCA when every feature is z-scored first.

In NumPy: `(X - X.mean(0)) / X.std(0)`

# 2 Two variables: covariance and correlation

## 2.1 Covariance

The *covariance* of two variables is the mean product of their deviations:

$$\text{cov}(x_1, x_2) = \frac{1}{n} \sum_i (x_1^{(i)} - \bar{x}_1)(x_2^{(i)} - \bar{x}_2) = \frac{1}{4}(4 + 4 - 1 - 1) = 1.5,$$

the four products being  $2 \cdot 2$ ,  $(-2)(-2)$ ,  $1 \cdot (-1)$  and  $(-1) \cdot 1$ . A positive covariance says that when one variable is above its mean the other tends to be too; here the two large points  $(2, 2)$  and  $(-2, -2)$  agree in sign and outweigh the two small ones that disagree. The covariance of a variable with itself is its variance.

## 2.2 Correlation

Covariance has the units of the two variables multiplied. Dividing by both standard deviations removes the units and gives the *Pearson correlation*,

$$r = \frac{\text{cov}(x_1, x_2)}{\sigma_1 \sigma_2} = \frac{1.5}{\sqrt{2.5}\sqrt{2.5}} = 0.6,$$

a number between  $-1$  and  $1$ . Equivalently,  $r$  is the covariance of the two z-scored variables, and (as the first worked-examples handout showed) it is the cosine of the angle between the two centred columns. On the lecture slide that plotted every pixel against its neighbour,  $r = 0.81$  for the photograph and  $-0.01$  for random pixels: that  $r$  is this formula applied to 4,032 pairs.

In NumPy: `np.cov(X.T, ddof=0)` gives the  $2 \times 2$  matrix; `np.corrcoef(X.T)` the correlations.

### 2.3 The covariance matrix

Put the variances on the diagonal and the covariances off it:

$$C = \begin{pmatrix} \text{var}(x_1) & \text{cov}(x_1, x_2) \\ \text{cov}(x_2, x_1) & \text{var}(x_2) \end{pmatrix} = \begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 2.5 \end{pmatrix}.$$

For  $d$  features  $C$  is  $d \times d$  and symmetric,  $C_{jk} = C_{kj}$ . The whole matrix is one product of the centred data with its own transpose,

$$C = \frac{1}{n} \tilde{X}^\top \tilde{X} = \frac{1}{n} \sum_{i=1}^n \tilde{\mathbf{x}}^{(i)} \tilde{\mathbf{x}}^{(i)\top},$$

which is the formula on the lecture's eigendecomposition slide. Check the  $(1, 2)$  entry:  $\frac{1}{4}(2 \cdot 2 + (-2)(-2) + 1 \cdot (-1) + (-1) \cdot 1) = 1.5$ . For a  $64 \times 64$  face image  $d = 4,096$  and  $C$  has sixteen million entries, which is why Section 5 computes PCA without ever forming it.

In NumPy: `C = Xc.T @ Xc / len(Xc)` (or `np.cov(Xc.T)`, which divides by  $n - 1$ ).

### 2.4 The variance along any direction

The one property of  $C$  that makes PCA work: for any unit vector  $\mathbf{w}$  ( $\|\mathbf{w}\| = 1$ ), the variance of the data projected onto  $\mathbf{w}$  is  $\mathbf{w}^\top C \mathbf{w}$ . The proof is two lines. The coordinate of point  $i$  along  $\mathbf{w}$  is  $z^{(i)} = \mathbf{w}^\top \tilde{\mathbf{x}}^{(i)}$ , a dot product; these coordinates have mean zero because the  $\tilde{\mathbf{x}}^{(i)}$  do; so

$$\text{var}(z) = \frac{1}{n} \sum_i (\mathbf{w}^\top \tilde{\mathbf{x}}^{(i)})^2 = \mathbf{w}^\top \left( \frac{1}{n} \sum_i \tilde{\mathbf{x}}^{(i)} \tilde{\mathbf{x}}^{(i)\top} \right) \mathbf{w} = \mathbf{w}^\top C \mathbf{w}.$$

**Example.** Along the first axis,  $\mathbf{w} = (1, 0)$ :  $\mathbf{w}^\top C \mathbf{w} = 2.5$ , the variance of  $x_1$ , as it should be. Along the diagonal,  $\mathbf{w} = (1, 1)/\sqrt{2}$ :

$$\mathbf{w}^\top C \mathbf{w} = \frac{1}{2} (1, 1) \begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 2.5 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{2} (1, 1) \cdot (4, 4) = 4.$$

Along the anti-diagonal,  $\mathbf{w} = (1, -1)/\sqrt{2}$ :  $\frac{1}{2} (1, -1) \cdot (1, -1) = 1$ . Check the first one directly: the coordinates along  $(1, 1)/\sqrt{2}$  are  $z = (4, -4, 0, 0)/\sqrt{2} = (2.83, -2.83, 0, 0)$ , whose mean square is  $(8 + 8)/4 = 4$ . The cloud is four times as spread along the diagonal as across it.

Try it. Compute  $\mathbf{w}^\top C \mathbf{w}$  for  $\mathbf{w} = (0, 1)$  and for  $\mathbf{w} = (0.6, 0.8)$ .

In NumPy: `w @ C @ w`, with  $\mathbf{w}$  a unit vector; `np.var(Xc @ w)`.

### 3 Eigenvectors and eigenvalues

#### 3.1 Definition, and the 2 x 2 case by hand

A vector  $v \neq 0$  is an *eigenvector* of a square matrix  $C$  if multiplying by  $C$  only rescales it:

$$Cv = \lambda v,$$

and the scale factor  $\lambda$  is its *eigenvalue*. Most vectors are turned by  $C$ ; eigenvectors are the ones that keep their direction.

For a  $2 \times 2$  matrix the eigenvalues can be found by hand.  $Cv = \lambda v$  means  $(C - \lambda I)v = 0$  with  $v \neq 0$ , which is only possible when the matrix  $C - \lambda I$  squashes some direction to zero, that is, when its determinant is zero:

$$\det \begin{pmatrix} 2.5 - \lambda & 1.5 \\ 1.5 & 2.5 - \lambda \end{pmatrix} = (2.5 - \lambda)^2 - 1.5^2 = 0 \implies 2.5 - \lambda = \pm 1.5 \implies \lambda_1 = 4, \lambda_2 = 1.$$

For  $\lambda_1 = 4$ , solve  $(C - 4I)v = 0$ : the first row reads  $-1.5v_1 + 1.5v_2 = 0$ , so  $v_1 = v_2$  and the unit eigenvector is  $v_1 = (1, 1)/\sqrt{2}$ . For  $\lambda_2 = 1$  the first row reads  $1.5v_1 + 1.5v_2 = 0$ , so  $v_2 = (1, -1)/\sqrt{2}$ . Check:

$$Cv_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 2.5 + 1.5 \\ 1.5 + 2.5 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 4 \\ 4 \end{pmatrix} = 4v_1, \quad Cv_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 2.5 - 1.5 \\ 1.5 - 2.5 \end{pmatrix} = 1 \cdot v_2.$$

These are exactly the two directions of Section 2.4, and the eigenvalues are exactly the variances found there: 4 along the diagonal, 1 across it. That is not a coincidence, and the next subsection says why.

Beyond  $2 \times 2$  nobody solves the polynomial; the computer does it numerically. Two things to know about the call: `np.linalg.eigh` is for symmetric matrices (the `h` is for Hermitian) and is faster and more accurate than the general `eig`, and it returns the eigenvalues in *ascending* order, so the columns have to be reversed to put the largest first. An eigenvector is only defined up to sign:  $-v_1$  satisfies the same equation, and different libraries, or the same library on different machines, may return either. Assignment 2's helper fixes the sign so that the largest entry of each eigenvector is positive.

In NumPy: `lam, V = np.linalg.eigh(C); lam, V = lam[::-1], V[:, ::-1]`

#### 3.2 Why the eigenvectors of C are the principal axes

Two facts about symmetric matrices, stated without proof: their eigenvalues are real, and their eigenvectors can be chosen *orthonormal*, unit length and mutually perpendicular. (Check:  $v_1 \cdot v_2 = \frac{1}{2}(1 \cdot 1 + 1 \cdot (-1)) = 0$ .) So the eigenvectors form a basis, and any unit vector can be written in it,  $w = a_1v_1 + a_2v_2 + \dots + a_dv_d$  with  $\sum_k a_k^2 = 1$ . Now compute the variance along  $w$  using  $Cv_k = \lambda_k v_k$  and the orthonormality:

$$w^T C w = \left( \sum_j a_j v_j \right)^T \left( \sum_k a_k \lambda_k v_k \right) = \sum_k \lambda_k a_k^2.$$

This is a weighted average of the eigenvalues with non-negative weights  $a_k^2$  that sum to one. Its largest possible value is the largest eigenvalue,  $\lambda_1$ , reached when all the weight sits on  $\mathbf{v}_1$  ( $a_1 = \pm 1$ , the rest zero); its smallest is  $\lambda_d$ . *The direction of greatest variance is the top eigenvector, and the variance along it is the top eigenvalue.* Among directions perpendicular to  $\mathbf{v}_1$  the same argument gives  $\mathbf{v}_2$ , and so on. That is the whole justification of PCA, in four lines.

**Example.**  $\mathbf{w} = (0.6, 0.8)$  has  $a_1 = \mathbf{w} \cdot \mathbf{v}_1 = 1.4/\sqrt{2} = 0.99$  and  $a_2 = \mathbf{w} \cdot \mathbf{v}_2 = -0.2/\sqrt{2} = -0.14$ , so  $\mathbf{w}^\top \mathbf{C} \mathbf{w} = 4(0.99)^2 + 1(0.14)^2 = 3.92 + 0.02 = 3.94$ , which you can confirm by the direct product.

### 3.3 Total variance, and the Pythagorean split

The sum of the variances of all the features,  $\text{tr } \mathbf{C} = \sum_j C_{jj}$ , is the *total variance*; it is also the mean squared distance of the centred points from the origin,  $\frac{1}{n} \sum_i \|\tilde{\mathbf{x}}^{(i)}\|^2$ . It equals the sum of the eigenvalues (a fact about every square matrix): here  $2.5 + 2.5 = 4 + 1 = 5$ .

Project every point onto  $\mathbf{w}$  and it splits into a part along  $\mathbf{w}$  and a perpendicular remainder. By Pythagoras the squared lengths add, and averaging over the cloud,

$$\underbrace{\frac{1}{n} \sum_i \|\tilde{\mathbf{x}}^{(i)}\|^2}_{\text{total, 5}} = \underbrace{\mathbf{w}^\top \mathbf{C} \mathbf{w}}_{\text{projected variance}} + \underbrace{\frac{1}{n} \sum_i \|\tilde{\mathbf{x}}^{(i)} - z^{(i)} \mathbf{w}\|^2}_{\text{reconstruction error}}.$$

The left side does not depend on  $\mathbf{w}$ . So whichever  $\mathbf{w}$  captures the most variance leaves the least error: *maximising projected variance and minimising reconstruction error are the same problem*, and  $\mathbf{v}_1$  solves both. For  $\mathbf{w} = \mathbf{v}_1$  the projected variance is 4 and the error is  $1 = \lambda_2$ ; for  $\mathbf{w} = (1, 0)$  they are 2.5 and 2.5.

## 4 Projection and reconstruction

### 4.1 Coordinates in the new basis

Collect the top  $K$  eigenvectors as the columns of  $\mathbf{V}_K$  ( $d \times K$ ). The *coordinates* of a point in the new basis, its PCA scores, are the  $K$  dot products

$$\mathbf{z} = \mathbf{V}_K^\top (\mathbf{x} - \boldsymbol{\mu}), \quad z_k = \mathbf{v}_k^\top (\mathbf{x} - \boldsymbol{\mu}).$$

With  $K = d$  this is a pure rotation: nothing is lost. With  $K = 1$  and our four points,  $\mathbf{z} = \mathbf{v}_1^\top \tilde{\mathbf{x}}$  gives  $(2.83, -2.83, 0, 0)$ : two points far out along the diagonal, two at the origin. Points 3 and 4, which differ from each other by  $(2, -2)$ , receive the same single coordinate, because that difference lies entirely along  $\mathbf{v}_2$ .

### 4.2 The way back, and what is lost

Reconstruction reverses the projection and adds the mean back:

$$\hat{\mathbf{x}} = \boldsymbol{\mu} + \mathbf{V}_K \mathbf{z} = \boldsymbol{\mu} + \sum_{k=1}^K z_k \mathbf{v}_k.$$

With  $K = 1$ :  $\hat{\mathbf{x}}^{(1)} = (3, 1) + 2.83(1, 1)/\sqrt{2} = (3, 1) + (2, 2) = (5, 3)$ , exact, because point 1 lay on the  $\mathbf{v}_1$  line to begin with. But  $\hat{\mathbf{x}}^{(3)} = (3, 1) + 0 = (3, 1)$ , while the true point is  $(4, 0)$ : the error vector is  $(1, -1)$ , of squared length 2. Point 4's error is  $(-1, 1)$ , also 2; points 1 and 2 have none. Mean squared error over the four points:  $(0 + 0 + 2 + 2)/4 = 1 = \lambda_2$ . In general the mean squared reconstruction error with  $K$  components is the sum of the discarded eigenvalues,  $\sum_{k>K} \lambda_k$ , which is what the lecture's "residual fraction" curves plot: the fraction  $\sum_{k>K} \lambda_k / \sum_k \lambda_k$ , here  $1/5 = 20\%$  for  $K = 1$ . The complementary number,  $\lambda_1 / \sum_k \lambda_k = 80\%$ , is the "variance explained" of the scree plot.

In NumPy: `Z = Xc @ V[:, :K]; X_hat = mu + Z @ V[:, :K].T`

Try it. Reconstruct point 2 with  $K = 1$  and with  $K = 2$ .

## 5 Computing PCA without forming C: the SVD

Every matrix has a *singular value decomposition*,  $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ , with  $\mathbf{U}$  ( $n \times r$ ) and  $\mathbf{V}$  ( $d \times r$ ) having orthonormal columns and  $\mathbf{S}$  a diagonal matrix of non-negative *singular values*  $s_1 \geq s_2 \geq \dots$ , where  $r$  is the rank of  $\tilde{\mathbf{X}}$ . Substitute it into the covariance:

$$\mathbf{C} = \frac{1}{n} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} = \frac{1}{n} \mathbf{V} \mathbf{S} \mathbf{U}^\top \mathbf{U} \mathbf{S} \mathbf{V}^\top = \mathbf{V} \left( \frac{1}{n} \mathbf{S}^2 \right) \mathbf{V}^\top,$$

using  $\mathbf{U}^\top \mathbf{U} = \mathbf{I}$ . Compare with the eigendecomposition  $\mathbf{C} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$ : the right singular vectors of the centred data *are* the eigenvectors of  $\mathbf{C}$ , and  $\lambda_k = s_k^2/n$  (or  $s_k^2/(n-1)$  with the other convention). This is how scikit-learn and Assignment 2's `fit_pca` compute PCA: the SVD of the  $n \times d$  data never builds the  $d \times d$  covariance, which for images would not fit in memory.

The rank is at most  $\min(n-1, d)$ : the  $-1$  is because centring removes one degree of freedom (the rows sum to zero). With four points in two dimensions,  $r \leq 2$ ; with 400 faces of 4,096 pixels, at most 399 eigenvalues are non-zero however many pixels there are, a fact that surprises people the first time a scree plot stops early.

In NumPy: `U, s, Vt = np.linalg.svd(Xc, full_matrices=False); lam = s**2 / len(Xc); V = Vt.T`

## 6 Standardization: the same points, different units

PCA maximises variance, and variance depends on units. Suppose the first feature had been recorded in different units, ten times larger:  $x_1 = 50, 10, 40, 20$ . Then  $\text{var}(x_1) = 250$ ,  $\text{cov}(x_1, x_2) = 15$ , and

$$\mathbf{C}' = \begin{pmatrix} 250 & 15 \\ 15 & 2.5 \end{pmatrix}, \quad \lambda_1 \approx 250.9, \quad \mathbf{v}_1 \approx (0.998, 0.060).$$

The first component now points almost exactly along the first feature and carries 99% of the variance: the second feature has become invisible, although the *shape* of the cloud, in the sense of which points are near which, has not changed at all. (Check the eigenvalues quickly:  $\lambda_1 + \lambda_2 = 252.5$  and  $\lambda_1 \lambda_2 = \det \mathbf{C}' = 625 - 225 = 400$ .)

Z-scoring each feature first (Section 1.3) gives every feature variance 1, so the covariance matrix of the z-scores is the *correlation matrix*,

$$\mathbf{C}_z = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0.6 \\ 0.6 & 1 \end{pmatrix}, \quad \lambda_1 = 1.6, \lambda_2 = 0.4, \mathbf{v}_1 = (1, 1)/\sqrt{2},$$

whatever the original units were (for a  $2 \times 2$  correlation matrix the eigenvalues are always  $1 \pm r$ ). Standardize when features are in different units or have variances that differ for uninteresting reasons; do not standardize when the differences in variance are the signal, as when every feature is a neuron's firing rate and some neurons really are more active. Assignment 1 standardizes the IT units before PCA for the first reason.

In NumPy: `Xz = (X - X.mean(0)) / X.std(0)`; then PCA as before.

## 7 Whitening: one step further

Projecting onto all  $d$  eigenvectors rotates the cloud so that its coordinates are *uncorrelated*: the covariance of  $\mathbf{z} = \mathbf{V}^\top \tilde{\mathbf{x}}$  is  $\mathbf{V}^\top \mathbf{C} \mathbf{V} = \mathbf{\Lambda}$ , diagonal, with the eigenvalues down the diagonal. Dividing coordinate  $k$  by  $\sqrt{\lambda_k}$  then gives every coordinate variance 1:

$$\mathbf{z}_w = \mathbf{\Lambda}^{-1/2} \mathbf{V}^\top (\mathbf{x} - \boldsymbol{\mu}), \quad \text{cov}(\mathbf{z}_w) = \mathbf{I}.$$

This is *whitening*: the cloud becomes a round ball of unit radius, every direction equally spread. On the four points, the whitened coordinates are  $z_1/\sqrt{4} = (1.41, -1.41, 0, 0)$  and  $z_2/\sqrt{1} = (0, 0, 1.41, -1.41)$ , and both have mean square 1. Whitening is what a learner wants as input when no direction should be privileged; it is also, to a first approximation, what the retina does to the correlations between neighbouring pixels. It is not the same as z-scoring: z-scoring rescales the original features, whitening rescales the principal coordinates, and only the second removes correlations.

In NumPy: `Zw = (Xc @ V) / np.sqrt(lam)`

## 8 All of it in code

The following function is, line for line, the PCA of Assignment 2 (its `fit_pca` helper divides by  $n - 1$  and fixes eigenvector signs; both are included). Run it on the four points and every number in this handout comes back, multiplied by  $4/3$  where the convention says so.

```

import numpy as np

X = np.array([[5, 3], [1, -1], [4, 0], [2, 2]], dtype=float) # (n, d)

def fit_pca(X):
    mean = X.mean(axis=0) # section 1.1
    centred = X - mean
    _, s, Vt = np.linalg.svd(centred, full_matrices=False) # section 6
    eigenvalues = s**2 / (len(X) - 1) # n - 1 convention
    eigenvectors = Vt.T # one unit eigenvector per column
    for j in range(eigenvectors.shape[1]): # deterministic sign
        if eigenvectors[np.argmax(np.abs(eigenvectors[:, j])), j] < 0:
            eigenvectors[:, j] *= -1
    return mean, eigenvalues, eigenvectors

mean, lam, V = fit_pca(X)
print(lam) # [5.333 1.333] = (4, 1) * 4/3
print(V[:, 0]) # [0.707 0.707] = v1
Z = (X - mean) @ V[:, :1] # section 5.1: one coordinate per point
X_hat = mean + Z @ V[:, :1].T # section 5.2: reconstruction with K = 1
print(((X - X_hat)**2).sum(1)) # [0. 0. 2. 2.]: residuals, point by point
print(lam[1:].sum() / lam.sum()) # 0.2: discarded fraction, either convention

```

Two habits worth keeping from this. Never compare eigenvalues across two analyses without checking they used the same  $n$  or  $n - 1$  convention, and never compare eigenvector signs across two analyses at all; compare the variance fractions and the absolute values of the projections instead.

### Answers to “Try it”

§2.4:  $\mathbf{w} = (0, 1)$  gives 2.5, the variance of  $x_2$ ;  $\mathbf{w} = (0.6, 0.8)$  gives  $0.36 \cdot 2.5 + 2 \cdot 0.48 \cdot 1.5 + 0.64 \cdot 2.5 = 0.9 + 1.44 + 1.6 = 3.94$ , matching §3.2. §4: point 2 has  $z_1 = -2.83$  and  $z_2 = 0$ , so  $\hat{\mathbf{x}}^{(2)} = (3, 1) - 2.83(1, 1)/\sqrt{2} = (1, -1)$  already with  $K = 1$ ;  $K = 2$  adds  $0 \cdot \mathbf{v}_2$  and changes nothing. Points 3 and 4 are the ones that need the second component.

### Symbols used in this handout

|                                                     |                                                       |                                              |                                                                                          |
|-----------------------------------------------------|-------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------|
| $\mathbf{X}, \tilde{\mathbf{X}}$                    | $n \times d$ data; the same, centred                  | $\mathbf{C}$                                 | $d \times d$ covariance matrix, $\frac{1}{n} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ |
| $\boldsymbol{\mu}, \bar{x}_j$                       | mean vector; mean of feature $j$                      | $\lambda_k, \mathbf{v}_k$                    | $k$ -th eigenvalue and unit eigenvector of $\mathbf{C}$                                  |
| $\sigma_j, \text{var}, \text{cov}$                  | standard deviation; variance; covariance              | $\mathbf{V}_K, \boldsymbol{\Lambda}$         | top $K$ eigenvectors; diagonal of eigenvalues                                            |
| $r$                                                 | Pearson correlation, $\text{cov}/(\sigma_1 \sigma_2)$ | $\mathbf{z}, z_k$                            | PCA coordinates (scores); the $k$ -th one                                                |
| $\mathbf{w}, \mathbf{w}^\top \mathbf{C} \mathbf{w}$ | a unit direction; the variance along it               | $\hat{\mathbf{x}}$                           | reconstruction, $\boldsymbol{\mu} + \mathbf{V}_K \mathbf{z}$                             |
| $\text{tr } \mathbf{C}$                             | total variance, $\sum_j C_{jj} = \sum_k \lambda_k$    | $\mathbf{U} \mathbf{S} \mathbf{V}^\top, s_k$ | the SVD; singular values                                                                 |